

INTRODUCTION TO DIGITAL SYSTEMS AND NUMBER SYSTEMS

Digital systems are the foundation of every modern electronic device. Whether we look at a personal computer, a smartphone, an automatic teller machine, a satellite, or even a microwave oven, all of them rely on digital logic to operate. The word “digital” comes from the Latin word *digitus*, meaning finger, because counting on fingers gives discrete values rather than continuous ones. In a digital system, every quantity is represented by a finite set of discrete values, most commonly the two values 0 and 1. These two values correspond to two distinct voltage levels in an electronic circuit, usually called LOW (0) and HIGH (1).

The study of digital logic design begins with understanding the difference between analog and digital quantities, the way numbers are represented inside a digital system, and how arithmetic and logical operations are performed using only 0s and 1s. Because computers cannot directly understand the decimal numbers we use in everyday life, all data — numbers, characters, images, sound, and instructions — must first be converted into binary form. This unit therefore introduces digital systems, the four most important number systems used in digital electronics, methods of converting numbers from one system to another, the rules of binary arithmetic, the concept of complements for representing negative numbers, and finally the various binary codes that are used in modern computers and communication systems.

DIGITAL SYSTEMS

A digital system is an electronic system that handles information in the form of discrete values. Unlike analog systems, where signals can take any value within a continuous range, digital systems work with only a finite number of distinct values, usually just two: 0 and 1. These two values represent the two stable states of an electronic switch — OFF and ON — and form the basis of binary logic. A digital system is built from basic building blocks such as logic gates, flip-flops, registers, counters, and memory units. By combining these building blocks in different ways, designers can construct devices ranging from simple calculators to highly complex microprocessors.

The reason digital systems are preferred over analog systems is that they are more reliable, more accurate, easier to design, and easier to maintain. A digital signal is either present (1) or absent (0); minor variations in voltage level due to noise do not change its meaning. This property, called noise immunity, is one of the strongest advantages of digital electronics. In addition, digital data can be stored, copied, and transmitted without any loss in quality, which is impossible in analog systems where every copy is slightly degraded compared to the original.

ANALOG VS DIGITAL SIGNALS

To understand digital systems clearly, we must first understand the difference between analog and digital signals. An analog signal is one whose amplitude varies continuously with time. Examples include the human voice, temperature, pressure, and the output of a microphone. A digital signal, on the other hand, has only two discrete amplitude levels — generally a low level representing 0 and a high level representing 1. Examples include the output of a logic gate, data transmitted on a USB cable, and the signal stored on a hard drive.

- **Continuous vs Discrete:** An analog signal can take any value within a given range, while a digital signal can take only specific values, typically 0 and 1.
- **Noise Effect:** Analog signals are highly affected by noise because every small change in voltage changes the value. Digital signals are far less affected because the receiver only needs to distinguish between HIGH and LOW.

- **Storage:** Analog data is difficult to store accurately for a long time and degrades each time it is copied. Digital data is stored as bits and can be copied infinitely without any loss.
- **Processing:** Analog signals are processed using components like resistors, capacitors, and operational amplifiers. Digital signals are processed using logic gates, flip-flops, and microprocessors.
- **Accuracy:** Digital systems give accuracy as high as desired by simply increasing the number of bits used to represent a quantity.
- **Cost and Size:** Modern digital integrated circuits are much smaller and cheaper than their analog counterparts due to advances in VLSI technology.

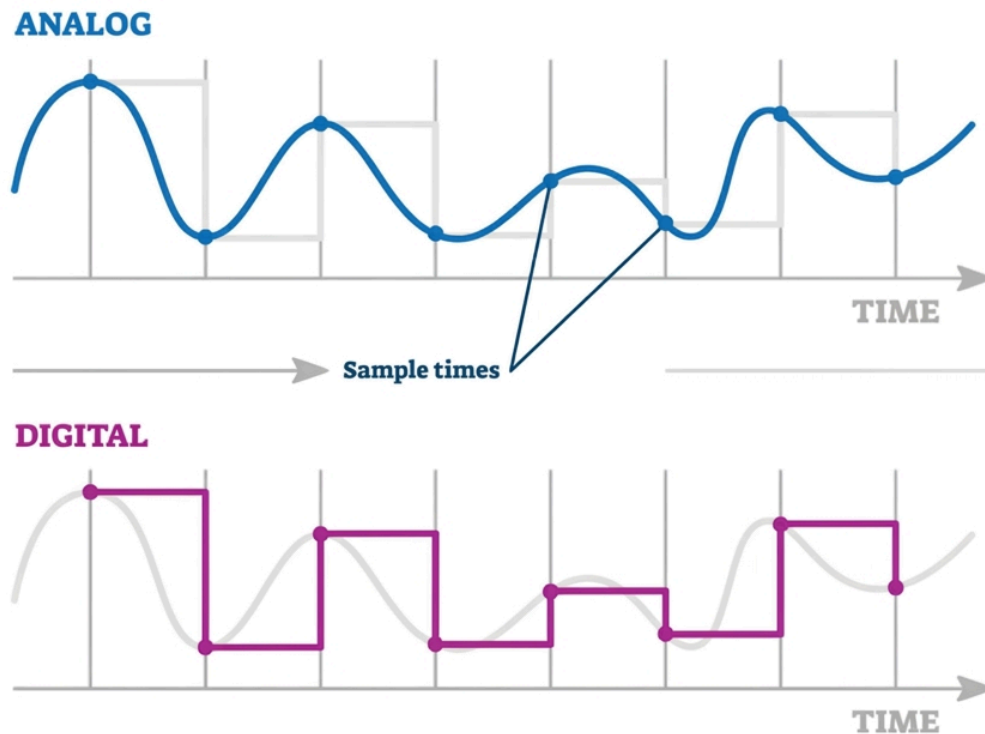


Figure 1: Comparison Between Analog and Digital Signals

ADVANTAGES OF DIGITAL SYSTEMS

Digital systems have replaced analog systems in most modern applications because of several important advantages. The major benefits are listed below.

- **Reliability:** Digital signals are immune to small variations in voltage, temperature, and component aging. As long as the signal can be recognized as either 0 or 1, the system works correctly.
- **Accuracy and Precision:** By using more bits, the precision of a digital system can be increased to any required level. For example, a 16-bit system gives much higher precision than an 8-bit system.
- **Easy Storage:** Digital data can be stored easily in memory devices such as RAM, ROM, hard disks, flash drives, and optical disks.
- **Easy Transmission:** Digital data can be transmitted over long distances using techniques such as error-correcting codes, encryption, and compression, without losing quality.

- **Programmability:** The same digital hardware can be programmed to perform many different tasks. A microprocessor, for example, can run a calculator, a word processor, or a game depending on the program loaded.
- **Reproducibility:** Digital data can be copied any number of times without any degradation. The hundredth copy is identical to the original.
- **Low Cost:** Due to the rapid development of integrated circuit technology, digital components have become very inexpensive and are available in large quantities.
- **Compatibility with Computers:** All modern computers and communication systems are digital, so digital signals can be processed by them directly without conversion.

LIMITATIONS OF DIGITAL SYSTEMS

Although digital systems offer many advantages, they also have some limitations that must be considered during design.

- **Conversion Required:** The real world is analog. Whenever a digital system has to deal with real-world signals like sound, temperature, or light, an analog-to-digital converter (ADC) is needed at the input and a digital-to-analog converter (DAC) is needed at the output.
- **Bandwidth:** Digital signals require greater bandwidth for transmission compared to analog signals carrying the same information.
- **Power Consumption:** Digital systems may consume more power than equivalent analog systems for very simple applications.
- **Complexity for Simple Tasks:** For very small and simple jobs, building a digital system may be unnecessarily complex compared to a small analog circuit.
- **Synchronization:** High-speed digital systems require precise timing and clock synchronization, which adds design complexity and can limit operating speed.

APPLICATIONS OF DIGITAL SYSTEMS

Digital systems are used in almost every field of modern life. Some important applications are mentioned below.

- **Computers and Communication:** Personal computers, laptops, servers, smartphones, the internet, satellite communication, mobile networks, and digital telephones.
- **Consumer Electronics:** Digital cameras, televisions, music players, video games, washing machines, microwave ovens, and air conditioners.
- **Industrial Control:** Programmable logic controllers (PLCs), robotics, automated manufacturing, process control, and sensor networks.
- **Medical Equipment:** Digital thermometers, blood-pressure monitors, ECG machines, ultrasound, MRI, CT scanners, and pacemakers.
- **Defense and Aerospace:** Radar, missile guidance systems, navigation, satellite imaging, and avionics in modern aircraft.
- **Banking and Commerce:** ATM machines, credit-card processing, online banking, e-commerce, and digital payment systems.
- **Transportation:** Engine control units in cars, GPS navigation, traffic light systems, and railway signaling.

NUMBER SYSTEMS

A number system is a mathematical notation used to represent numbers using a particular set of symbols, called digits. Every number system has a base or radix, which is the total number of distinct

symbols (digits) used in that system. The position of each digit in a number determines its weight, and the total value of the number is obtained by summing the products of each digit and its corresponding weight. This kind of representation is called positional notation, and almost all modern number systems are positional.

In digital electronics, four number systems are most commonly used: the decimal system (base 10) which is familiar from everyday life; the binary system (base 2) which is the natural language of digital circuits; the octal system (base 8) which is a compact way of representing binary; and the hexadecimal system (base 16) which is widely used in computer programming and memory addressing. Each of these systems is described in detail below.

1. Decimal Number System (Base 10)

The decimal number system is the one we use in our daily lives. It has a base of 10, which means it uses ten distinct digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. The system is also called the denary system. The position of each digit determines its weight, which is a power of 10. The rightmost digit has weight $10^0 = 1$, the next digit to the left has weight $10^1 = 10$, the next has weight $10^2 = 100$, and so on. For numbers with a fractional part, the weights to the right of the decimal point are 10^{-1} , 10^{-2} , 10^{-3} , and so on.

Example 1: Express the decimal number 8754 using positional notation.

$$\begin{aligned} 8754 &= 8 \times 10^3 + 7 \times 10^2 + 5 \times 10^1 + 4 \times 10^0 \\ &= 8000 + 700 + 50 + 4 = 8754 \end{aligned}$$

Example 2: Express the decimal number 327.45 using positional notation.

$$\begin{aligned} 327.45 &= 3 \times 10^2 + 2 \times 10^1 + 7 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} \\ &= 300 + 20 + 7 + 0.4 + 0.05 = 327.45 \end{aligned}$$

2. Binary Number System (Base 2)

The binary number system is the most important number system in digital electronics. It has a base of 2 and uses only two digits: 0 and 1. Each digit in a binary number is called a bit, which is short for binary digit. A group of 4 bits is called a nibble, a group of 8 bits is called a byte, and a group of 16, 32, or 64 bits is called a word, depending on the computer architecture. The reason the binary system is used in digital circuits is that an electronic switch (such as a transistor) has only two stable states — ON and OFF — which can naturally represent 1 and 0.

The position of each bit in a binary number determines its weight, which is a power of 2. The rightmost bit, called the Least Significant Bit (LSB), has weight $2^0 = 1$, the next bit has weight $2^1 = 2$, the next has weight $2^2 = 4$, and so on. The leftmost bit is called the Most Significant Bit (MSB).

Example 1: Convert the binary number 1101_2 into decimal.

$$\begin{aligned} 1101_2 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 8 + 4 + 0 + 1 = 13_{10} \end{aligned}$$

Example 2: Convert the binary number 10110.101_2 into decimal.

$$\begin{aligned} 10110.101_2 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ &= 16 + 0 + 4 + 2 + 0 + 0.5 + 0 + 0.125 = 22.625_{10} \end{aligned}$$

3. Octal Number System (Base 8)

The octal number system has a base of 8 and uses eight digits: 0, 1, 2, 3, 4, 5, 6, and 7. The octal system was developed as a compact way of representing binary numbers, because each octal digit corresponds exactly to three binary digits. This makes conversion between binary and octal extremely simple. Although octal is less commonly used today than hexadecimal, it is still used in some operating systems and in setting file permissions in UNIX-based systems.

The position of each digit in an octal number determines its weight, which is a power of 8. The rightmost digit has weight $8^0 = 1$, the next has weight $8^1 = 8$, and so on.

Example 1: Convert the octal number 354_8 into decimal.

$$\begin{aligned} 354_8 &= 3 \times 8^2 + 5 \times 8^1 + 4 \times 8^0 \\ &= 192 + 40 + 4 = 236_{10} \end{aligned}$$

Example 2: Convert the octal number 27.4_8 into decimal.

$$\begin{aligned} 27.4_8 &= 2 \times 8^1 + 7 \times 8^0 + 4 \times 8^{-1} \\ &= 16 + 7 + 0.5 = 23.5_{10} \end{aligned}$$

4. Hexadecimal Number System (Base 16)

The hexadecimal number system has a base of 16 and uses sixteen distinct symbols. The first ten symbols are the digits 0 through 9 (with their usual values), and the remaining six are the letters A, B, C, D, E, and F, which represent the values 10, 11, 12, 13, 14, and 15 respectively. The hexadecimal system is widely used in computing because each hexadecimal digit corresponds to exactly four binary digits, so a long binary number can be written compactly in hexadecimal form. Hexadecimal numbers are commonly used to represent memory addresses, color codes (in HTML and CSS), MAC addresses, and machine-language instructions.

The position of each digit in a hexadecimal number determines its weight, which is a power of 16. The rightmost digit has weight $16^0 = 1$, the next has weight $16^1 = 16$, the next has weight $16^2 = 256$, and so on.

Example 1: Convert the hexadecimal number $2B5_{16}$ into decimal.

$$\begin{aligned} 2B5_{16} &= 2 \times 16^2 + 11 \times 16^1 + 5 \times 16^0 \\ &= 512 + 176 + 5 = 693_{10} \end{aligned}$$

Example 2: Convert the hexadecimal number $1A.C_{16}$ into decimal.

$$\begin{aligned} 1A.C_{16} &= 1 \times 16^1 + 10 \times 16^0 + 12 \times 16^{-1} \\ &= 16 + 10 + 0.75 = 26.75_{10} \end{aligned}$$

Decimal	Binary	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9

10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Table 2: Equivalent Values in Decimal, Binary, Octal and Hexadecimal

NUMBER SYSTEM CONVERSIONS

Since digital systems work in binary while humans normally use decimal, conversion between number systems is one of the most basic skills in digital logic design. There are well-defined methods for converting any number from one system to another. These methods are based on the positional weights of the digits and the repeated-division and repeated-multiplication techniques. Each major conversion type is explained in detail below with worked examples.

DECIMAL TO BINARY CONVERSION

To convert a decimal integer into binary, the method of repeated division by 2 is used. The decimal number is divided by 2, and the remainder (which will be either 0 or 1) is recorded. The quotient obtained is again divided by 2, and the new remainder is recorded. This process is continued until the quotient becomes zero. The binary equivalent is then obtained by writing the remainders in reverse order — that is, the last remainder is the most significant bit (MSB) and the first remainder is the least significant bit (LSB).

To convert the fractional part of a decimal number into binary, the method of repeated multiplication by 2 is used. The fractional part is multiplied by 2, and the integer part of the result (either 0 or 1) is recorded as the next binary digit. The new fractional part is again multiplied by 2, and the process continues until the fractional part becomes zero or the desired precision is reached.

Example 1: Convert 45_{10} into binary.

$$\begin{array}{llll}
 45 \div 2 = 22 & \text{remainder } 1 & (\text{LSB}) \\
 22 \div 2 = 11 & \text{remainder } 0 \\
 11 \div 2 = 5 & \text{remainder } 1 \\
 5 \div 2 = 2 & \text{remainder } 1 \\
 2 \div 2 = 1 & \text{remainder } 0 \\
 1 \div 2 = 0 & \text{remainder } 1 & (\text{MSB})
 \end{array}$$

Reading remainders bottom to top: $45_{10} = 101101_2$

Example 2: Convert 0.625_{10} into binary.

$$\begin{array}{llll}
 0.625 \times 2 = 1.250 & \text{integer part } = 1 \\
 0.250 \times 2 = 0.500 & \text{integer part } = 0 \\
 0.500 \times 2 = 1.000 & \text{integer part } = 1
 \end{array}$$

Reading integer parts top-down: $0.625_{10} = 0.101_2$

Example 3: Convert 25.375_{10} into binary.

Integer part: $25_{10} = 11001_2$ Fractional part: $0.375_{10} = 0.011_2$
 Combined result: $25.375_{10} = 11001.011_2$

BINARY TO DECIMAL CONVERSION

To convert a binary number into decimal, multiply each bit by its positional weight (a power of 2) and add all the products. For the integer part, weights are $2^0, 2^1, 2^2, \dots$ starting from the right. For the fractional part, weights are $2^{-1}, 2^{-2}, 2^{-3}, \dots$ starting from the binary point.

Example 1: Convert 110101_2 into decimal.

$$\begin{aligned} 110101_2 &= 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 32 + 16 + 0 + 4 + 0 + 1 = 53_{10} \end{aligned}$$

Example 2: Convert 1011.011_2 into decimal.

$$1011.011_2 = 8 + 0 + 2 + 1 + 0 + 0.25 + 0.125 = 11.375_{10}$$

DECIMAL TO OCTAL CONVERSION

The decimal-to-octal conversion uses the same idea as decimal-to-binary, but division is performed by 8 instead of 2. The remainders, read from bottom to top, give the octal equivalent of the integer part. For the fractional part, multiplication by 8 is used, and the integer parts of successive products give the octal digits.

Example 1: Convert 365_{10} into octal.

$$\begin{aligned} 365 \div 8 &= 45 \quad \text{remainder } 5 \quad (\text{LSD}) \\ 45 \div 8 &= 5 \quad \text{remainder } 5 \\ 5 \div 8 &= 0 \quad \text{remainder } 5 \quad (\text{MSD}) \end{aligned}$$

Result: $365_{10} = 555_8$

Example 2: Convert 0.625_{10} into octal.

$$\begin{aligned} 0.625 \times 8 &= 5.000 \quad \text{integer part} = 5 \\ \text{Result: } 0.625_{10} &= 0.5_8 \end{aligned}$$

OCTAL TO DECIMAL CONVERSION

To convert an octal number into decimal, multiply each octal digit by its positional weight (a power of 8) and add the results.

Example: Convert 627_8 into decimal.

$$627_8 = 6 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 = 384 + 16 + 7 = 407_{10}$$

DECIMAL TO HEXADECIMAL CONVERSION

The decimal-to-hexadecimal conversion follows the same divide-by-base approach, but using base 16. The remainders are between 0 and 15, where the values 10 to 15 are written as A to F. For fractional parts, repeated multiplication by 16 is used.

Example 1: Convert 957_{10} into hexadecimal.

$$\begin{aligned} 957 \div 16 &= 59 \quad \text{remainder } 13 \text{ (D)} \quad (\text{LSD}) \\ 59 \div 16 &= 3 \quad \text{remainder } 11 \text{ (B)} \\ 3 \div 16 &= 0 \quad \text{remainder } 3 \quad (\text{MSD}) \end{aligned}$$

Result: $957_{10} = 3BD_{16}$

Example 2: Convert 0.6875_{10} into hexadecimal.

$$\begin{aligned} 0.6875 \times 16 &= 11.0 \quad \text{integer part} = 11 \text{ (B)} \\ \text{Result: } 0.6875_{10} &= 0.B_{16} \end{aligned}$$

HEXADECIMAL TO DECIMAL CONVERSION

To convert a hexadecimal number into decimal, multiply each hexadecimal digit by its positional weight (a power of 16) and add the results, remembering that A=10, B=11, C=12, D=13, E=14, F=15.

Example: Convert 4F2A₁₆ into decimal.

$$\begin{aligned} 4F2A_{16} &= 4 \times 16^3 + 15 \times 16^2 + 2 \times 16^1 + 10 \times 16^0 \\ &= 16384 + 3840 + 32 + 10 = 20266_{10} \end{aligned}$$

BINARY TO OCTAL CONVERSION

Since $2^3 = 8$, each octal digit corresponds to exactly three binary bits. To convert a binary number into octal, group the binary bits into sets of three, starting from the binary point and moving outwards (rightwards for the integer part, leftwards for the fractional part). If the leftmost or rightmost group has fewer than three bits, add zeros to complete it. Then replace each group of three bits with its octal equivalent.

Example: Convert 1011010.1101₂ into octal.

$$\begin{aligned} \text{Integer part:} \quad & 001 \ 011 \ 010 \rightarrow 1 \ 3 \ 2 \\ \text{Fractional part:} \quad & 110 \ 100 \rightarrow 6 \ 4 \\ \text{Result:} \quad & 1011010.1101_2 = 132.64_8 \end{aligned}$$

OCTAL TO BINARY CONVERSION

To convert an octal number into binary, simply replace each octal digit with its 3-bit binary equivalent.

Example: Convert 472₈ into binary.

$$\begin{aligned} 4 &= 100, \quad 7 = 111, \quad 2 = 010 \\ \text{Result:} \quad & 472_8 = 100111010_2 \end{aligned}$$

BINARY TO HEXADECIMAL CONVERSION

Since $2^4 = 16$, each hexadecimal digit corresponds to exactly four binary bits. To convert binary into hexadecimal, group the binary bits into sets of four, starting from the binary point and moving outwards. Add zeros if necessary to complete the leftmost or rightmost groups. Then replace each group with its hexadecimal equivalent.

Example: Convert 110110111011.10101₂ into hexadecimal.

$$\begin{aligned} \text{Integer part:} \quad & 1101 \ 1011 \ 1011 \rightarrow D \ B \ B \\ \text{Fractional part:} \quad & 1010 \ 1000 \rightarrow A \ 8 \\ \text{Result:} \quad & 110110111011.10101_2 = DBB.A8_{16} \end{aligned}$$

HEXADECIMAL TO BINARY CONVERSION

To convert a hexadecimal number into binary, simply replace each hex digit with its 4-bit binary equivalent.

Example: Convert 7AF₁₆ into binary.

$$\begin{aligned} 7 &= 0111, \quad A = 1010, \quad F = 1111 \\ \text{Result:} \quad & 7AF_{16} = 011110101111_2 \end{aligned}$$

OCTAL TO HEXADECIMAL AND HEXADECIMAL TO OCTAL

There is no direct method for converting between octal and hexadecimal. The standard approach is to first convert the given number into binary (since binary is the common bridge), and then convert the binary number into the target base.

Example: Convert 745₈ into hexadecimal.

Step 1 – Octal to binary: 7 = 111, 4 = 100, 5 = 101 → 111100101₂

Step 2 – Binary to hex: 0001 1110 0101 → 1 E 5

Result: 745₈ = 1E5₁₆

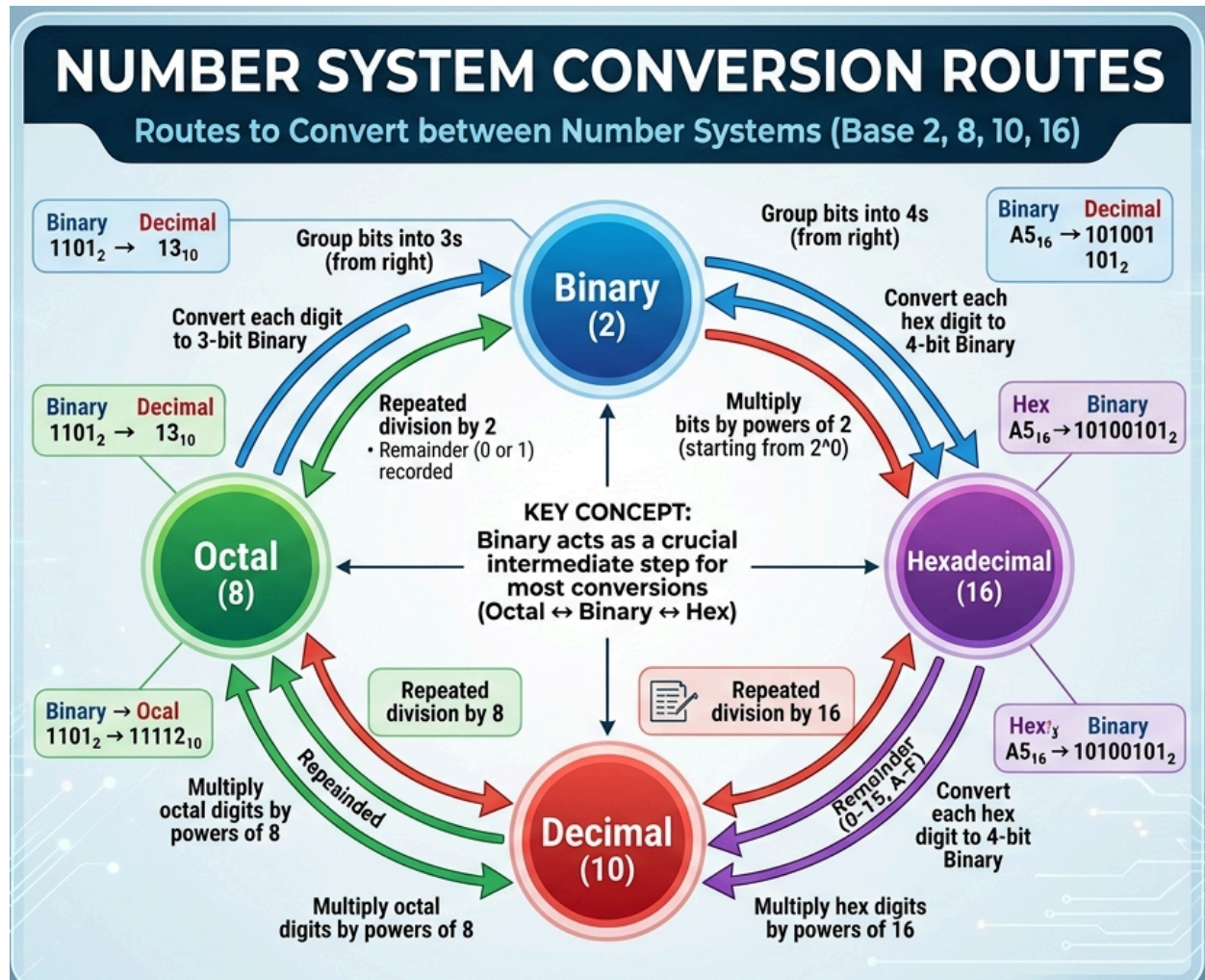


Figure 1.2: Number System Conversion Routes

BINARY ARITHMETIC

All arithmetic operations performed inside a digital computer are carried out in binary form. Although decimal arithmetic is more familiar to humans, binary arithmetic is much simpler for machines because each digit can be only 0 or 1, and the rules are very few. The four basic arithmetic operations — addition, subtraction, multiplication, and division — follow the same general procedures used in decimal arithmetic, but the rules of carry, borrow, and place value apply to base 2 instead of base 10.

BINARY ADDITION

Binary addition is the most fundamental of all binary operations because subtraction, multiplication, and even division can ultimately be implemented using addition. There are only four basic rules for adding two binary digits.

➤ $0 + 0 = 0$

- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10$ (sum of 0 with a carry of 1 to the next column)

When more than two bits are added, or when carries are involved, a fifth case appears: $1 + 1 + 1 = 11$ (a sum of 1 and a carry of 1). The procedure for adding multi-bit binary numbers is exactly the same as for decimal addition: align the numbers by their least-significant digits, add column by column from right to left, and propagate any carry to the next column.

Example 1: Add 1011_2 and 1101_2 .

```

      1 1 1           ← carries
      1 0 1 1
    + 1 1 0 1
    -----
      1 1 0 0 0      result
Verification:  $1011_2 = 11$ ,  $1101_2 = 13$ ,  $\text{sum} = 24 = 11000_2$  ✓

```

Example 2: Add 11110_2 and 10111_2 .

```

      1 1 1 1 0
    + 1 0 1 1 1
    -----
      1 1 0 1 0 1
Verification:  $30 + 23 = 53 = 110101_2$  ✓

```

BINARY SUBTRACTION

Binary subtraction also has only four basic rules. When the bit being subtracted is greater than the bit it is being subtracted from, a borrow is taken from the next higher column, just as in decimal subtraction.

- $0 - 0 = 0$
- $1 - 0 = 1$
- $1 - 1 = 0$
- $0 - 1 = 1$ (with a borrow of 1 from the next higher bit)

Example 1: Subtract 0110_2 from 1101_2 .

```

      1 1 0 1
    - 0 1 1 0
    -----
      0 1 1 1
Verification:  $13 - 6 = 7 = 0111_2$  ✓

```

Example 2: Subtract 10110_2 from 11010_2 .

```

      1 1 0 1 0
    - 1 0 1 1 0
    -----
      0 0 1 0 0
Verification:  $26 - 22 = 4 = 100_2$  ✓

```

BINARY MULTIPLICATION

Binary multiplication is much simpler than decimal multiplication because the multiplication table contains only four entries, all of which are obvious.

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

The procedure for multiplying multi-bit binary numbers is the same as for decimal multiplication: each bit of the multiplier is multiplied by the entire multiplicand, the partial products are shifted appropriately, and finally the partial products are added. Because each bit of the multiplier is either 0 or 1, the partial product is either zero or a copy of the multiplicand.

Example: Multiply 1101_2 by 101_2 .

```

      1 1 0 1      multiplicand
    ×   1 0 1      multiplier
    -----
      1 1 0 1      ← 1101 × 1
    0 0 0 0 0      ← 1101 × 0, shifted left 1
  1 1 0 1 0 0      ← 1101 × 1, shifted left 2
  -----
  1 0 0 0 0 0 1      final product
Verification:  $13 \times 5 = 65 = 1000001_2$  ✓

```

BINARY DIVISION

Binary division uses the same long-division procedure as decimal division. The divisor is compared with the leftmost portion of the dividend; if it fits, a 1 is placed in the quotient and the divisor is subtracted; if not, a 0 is placed and the next bit of the dividend is brought down. The process continues until all bits of the dividend have been used.

Example: Divide 11011_2 by 11_2 .

```

      1 0 0 1      quotient
    -----
  1 1 ) 1 1 0 1 1
      - 1 1
      -----
        0 0 1 1
        - 1 1
        -----
          0 0      remainder
Verification:  $27 \div 3 = 9 = 1001_2$  ✓

```

COMPLEMENTS OF NUMBERS

Complements are used in digital computers for two main purposes: simplifying the subtraction operation, and representing negative numbers. The advantage of using complements is that subtraction can be performed using the same hardware as addition, which makes the design of arithmetic circuits much simpler. For each base r , two types of complements are defined: the $(r-1)$'s complement, called the diminished radix complement, and the r 's complement, called the radix complement. In the binary system these are known as the 1's complement and the 2's complement. Similarly, in the decimal system they are called the 9's complement and the 10's complement.

1. 1's Complement of a Binary Number

The 1's complement of a binary number is obtained by inverting (flipping) every bit — that is, by changing every 0 to 1 and every 1 to 0. This is also called the bitwise NOT or bit complement of the number. The 1's complement can be obtained electronically by passing the number through a row of NOT gates.

Example 1: Find the 1's complement of 10110100_2 .

Inverting each bit: 01001011_2

Example 2: Find the 1's complement of 11111111_2 .

Inverting each bit: 00000000_2

2. 2's Complement of a Binary Number

The 2's complement of a binary number is obtained by adding 1 to its 1's complement. Alternatively, the 2's complement can be obtained directly by leaving all the trailing 0s and the rightmost 1 unchanged, and then inverting all bits to the left of that rightmost 1. The 2's complement is the standard method used by modern computers to represent negative numbers, because it gives a single representation for zero and allows arithmetic operations to be performed easily.

Example 1: Find the 2's complement of 10110100_2 .

Step 1 – 1's complement: 01001011

Step 2 – Add 1: $01001011 + 1 = 01001100$

Result: 2's complement = 01001100_2

Example 2: Find the 2's complement of 110100_2 (shortcut).

Rightmost 1 is in position 3; keep "100" unchanged, invert the rest.

$110100 \rightarrow 001100$

Result: 2's complement = 001100_2

3. 9's and 10's Complements (for Decimal Numbers)

For decimal numbers, the 9's complement of a number is obtained by subtracting each digit from 9. The 10's complement is obtained by adding 1 to the 9's complement. These are useful for performing decimal subtraction using addition.

Example: Find the 9's and 10's complements of the decimal number 4567.

9's complement: $(9999 - 4567) = 5432$

10's complement: $5432 + 1 = 5433$

SUBTRACTION USING 1'S COMPLEMENT

To subtract using 1's complement, the 1's complement of the subtrahend is added to the minuend. If a carry is generated from the most significant bit, it is added back into the result (called end-around carry) and the answer is positive. If no carry is generated, the result is negative and is in 1's complement form, so it must be re-complemented to find its magnitude.

Example: Subtract 0101_2 from 1010_2 using 1's complement.

1's complement of $0101 = 1010$

Add: $1010 + 1010 = 1\ 0100$

End-around carry: $0100 + 1 = 0101$

Result: $1010 - 0101 = 0101_2$ ✓

SUBTRACTION USING 2'S COMPLEMENT

To subtract using 2's complement, the 2's complement of the subtrahend is added to the minuend. If a carry is generated from the most significant bit, it is simply discarded and the answer is positive. If no

carry is generated, the result is negative and is in 2's complement form, so its 2's complement gives the actual magnitude. This method is used in almost all modern processors.

Example 1: Subtract 0110_2 from 1011_2 using 2's complement.

2's complement of $0110 = 1010$
Add: $1011 + 1010 = 1\ 0101$ (discard the carry)
Result: $1011 - 0110 = 0101_2 = 5_{10}$ (positive) ✓

Example 2: Subtract 1100_2 from 0101_2 using 2's complement.

2's complement of $1100 = 0100$
Add: $0101 + 0100 = 1001$ (no carry → result is negative)
2's complement of $1001 = 0111$ → magnitude is 7
Result: $0101 - 1100 = -0111_2 = -7_{10}$ ✓

REPRESENTATION OF SIGNED BINARY NUMBERS

Computers use a fixed number of bits (typically 8, 16, 32, or 64) to store each number. Among these bits, the leftmost one is generally reserved as the sign bit: 0 indicates a positive number, and 1 indicates a negative number. Three different representations are commonly used for signed numbers.

- **Sign-Magnitude Form:** The sign bit is followed by the magnitude in normal binary. For example, in 8 bits, +25 is 00011001 and -25 is 10011001. This form is simple but has two representations of zero (+0 and -0) and is rarely used in modern hardware.
- **1's Complement Form:** Positive numbers are stored in normal binary, while negative numbers are stored as the 1's complement of their magnitude. For example, +25 is 00011001 and -25 is 11100110. This form also has two representations of zero.
- **2's Complement Form:** Positive numbers are stored in normal binary, while negative numbers are stored as the 2's complement of their magnitude. For example, +25 is 00011001 and -25 is 11100111. This is the standard form used in modern computers because it has a unique zero and simplifies arithmetic.

BINARY CODES

A binary code is a way of representing information — numbers, characters, instructions, or any other data — using sequences of binary digits. Different codes are used for different purposes in digital systems, depending on whether the data needs to be displayed, processed, transmitted, or stored. Some codes are designed for arithmetic, others for error detection and correction, and others for representing alphanumeric characters. The most important binary codes used in digital electronics and computers are described below.

BCD (BINARY-CODED DECIMAL) CODE

BCD is the most widely used decimal code. In BCD, each decimal digit (0 through 9) is represented by its 4-bit binary equivalent. The most common form is the 8421 BCD code, where the four positional weights are 8, 4, 2, and 1. Although BCD uses 4 bits, only 10 out of the possible 16 combinations are valid; the combinations 1010 to 1111 are unused. BCD is widely used in digital instruments, calculators, digital clocks, and 7-segment displays because it allows decimal numbers to be displayed in their natural form without conversion.

Example: Represent the decimal number 459 in BCD.

$4 = 0100, \quad 5 = 0101, \quad 9 = 1001$
 $459_{10} = 0100\ 0101\ 1001$ (BCD)

Note that BCD is different from straight binary. The number 459 in pure binary is 111001011, while in BCD it is 0100 0101 1001. BCD always uses 4 bits per digit, regardless of the value.

EXCESS-3 CODE

The Excess-3 code, also known as XS-3, is obtained by adding 3 (0011 in binary) to each digit of the BCD code. It is a self-complementing code, which means that the 9's complement of any decimal digit can be obtained by simply inverting the bits of its Excess-3 representation. This property makes Excess-3 very useful in arithmetic operations, especially decimal subtraction. For example, the decimal digit 4 in Excess-3 is 0111; its 9's complement is 5, which in Excess-3 is 1000 — exactly the bitwise inversion of 0111.

Example: Represent the decimal number 596 in Excess-3 code.

$$5 + 3 = 8 = 1000, \quad 9 + 3 = 12 = 1100, \quad 6 + 3 = 9 = 1001$$
$$596_{10} = 1000 \ 1100 \ 1001 \quad (\text{Excess-3})$$

GRAY CODE

The Gray code, also called the reflected binary code, is a non-weighted code in which two successive numbers differ by only one bit. Because only one bit changes between consecutive values, the Gray code is very useful in applications where multiple bits change simultaneously could cause errors — for example, in shaft-position encoders, Karnaugh maps, and digital communication. Gray code is not used for arithmetic operations because it is non-weighted; that is, the position of a bit does not represent a fixed weight.

To convert binary into Gray code, the most significant bit is copied as it is. Each remaining Gray bit is obtained by performing the XOR (exclusive-OR) of the current binary bit and the previous binary bit. To convert Gray code back into binary, the most significant bit is again copied. Each remaining binary bit is obtained by XORing the corresponding Gray bit with the previously computed binary bit.

Example: Convert 1011_2 into Gray code.

$$\text{MSB: } 1 \quad (\text{copied})$$
$$1 \oplus 0 = 1, \quad 0 \oplus 1 = 1, \quad 1 \oplus 1 = 0$$
$$\text{Result: } 1011_2 = 1110 \quad (\text{Gray})$$

ASCII CODE

The American Standard Code for Information Interchange (ASCII) is a 7-bit alphanumeric code used to represent characters, digits, punctuation marks, and control symbols inside computers. Because it uses 7 bits, it can represent 128 different symbols. The first 32 codes (0 to 31) are control characters such as carriage return, line feed, and bell, while codes 32 to 127 represent printable characters. For example, the character A is represented by 1000001, the character a by 1100001, and the digit 0 by 0110000.

Extended ASCII uses 8 bits, allowing 256 different characters. The additional 128 codes are used for graphical characters, accented letters, and symbols from various languages. ASCII has been replaced by Unicode in most modern systems, but it remains the basis of many communication protocols and programming languages.

UNICODE

Unicode is a universal character encoding standard that aims to represent every character used in every written language in the world. Unlike ASCII, which is limited to 128 characters, Unicode can represent more than a million characters. The most common Unicode encoding is UTF-8, which uses variable-length encoding ranging from 1 to 4 bytes per character; characters in the ASCII range are represented by a single byte, ensuring backward compatibility. Other common encodings are UTF-16 and UTF-32. Unicode is the standard used by all modern operating systems, web browsers, and programming languages.

Decimal	BCD	Excess-3 (XS-3)	Gray Code
0	0000	0011	0000
1	0001	0100	0001
2	0010	0101	0011
3	0011	0110	0010
4	0100	0111	0110
5	0101	1000	0111
6	0110	1001	0101
7	0111	1010	0100
8	1000	1011	1100
9	1001	1100	1101

Table 2: BCD, Excess-3, and Gray Code Equivalents of Decimal Digits

ERROR DETECTION AND CORRECTION CODES

During data transmission and storage, errors may occur due to electrical noise, hardware faults, or external interference. An error means that one or more bits change their value (a 0 becomes a 1 or a 1 becomes a 0) without the receiver knowing about it. To overcome this problem, special codes called error detection codes and error correction codes are used. These codes add extra bits, called redundant or check bits, to the original data. The receiver uses these extra bits to verify whether the received data is correct, and in some cases to correct it without re-transmission.

PARITY BIT

The parity bit is the simplest method of error detection. A single extra bit is appended to the data so that the total number of 1s in the resulting codeword is either always even (even parity) or always odd (odd parity). At the receiver end, the total number of 1s is counted. If the parity rule is violated, an error is detected; otherwise, the data is assumed to be correct. The parity method can detect any odd number of bit errors but cannot detect an even number of errors, nor can it tell which bit was in error.

- **Even Parity:** The parity bit is set so that the total number of 1s, including the parity bit, is even. For example, the data 1011001 has four 1s, so the even parity bit is 0; the transmitted word is 10110010.
- **Odd Parity:** The parity bit is set so that the total number of 1s, including the parity bit, is odd. For the same data 1011001 with four 1s, the odd parity bit is 1; the transmitted word is 10110011.

CHECKSUM

A checksum is a value computed from a block of data and transmitted along with it. The receiver computes the checksum of the received data and compares it with the transmitted checksum. If they differ, an error is detected. Checksums are widely used in network communication and file transfer protocols. Common checksum algorithms include CRC (Cyclic Redundancy Check), MD5, and SHA variants.

HAMMING CODE

The Hamming code is an error-detecting and error-correcting code developed by Richard Hamming in 1950. Unlike a single parity bit, the Hamming code uses several parity bits placed at positions that are powers of two (positions 1, 2, 4, 8, ...). Each parity bit checks a specific group of data bits. By analyzing which parity checks fail at the receiver, the position of a single-bit error can be exactly determined and corrected. The Hamming code can detect up to two-bit errors and correct any single-bit error. It is widely used in computer memory systems (such as ECC RAM) and in digital communication.

For a data word of m bits, the number of parity bits p needed for the Hamming code satisfies the condition $2^p \geq m + p + 1$. For example, for a 4-bit data word, at least 3 parity bits are required, giving a 7-bit Hamming codeword (4 data bits + 3 parity bits).

REFERENCES

1. M. Morris Mano, Digital Logic and Computer Design, 1st Edition, Pearson, 1979, ISBN: 0132145103.
2. Thomas L. Floyd, Digital Fundamentals, 10th Edition, Prentice Hall, 2008, ISBN: 0132359235.
3. Stephen Brown and Zvonko Vranesic, Fundamentals of Digital Logic with Verilog Design, 2nd Edition, McGraw-Hill, 2008.
4. R. P. Jain, Modern Digital Electronics, 4th Edition, McGraw-Hill, 2009.
5. A. P. Malvino and D. P. Leach, Digital Principles and Applications, 8th Edition, McGraw-Hill, 2014.

REVIEW QUESTIONS

- Q1. Define a digital system. List the main differences between an analog signal and a digital signal.
- Q2. Discuss in detail the advantages and limitations of digital systems over analog systems.
- Q3. List at least five practical applications of digital systems in everyday life.
- Q4. What is a number system? Explain the concept of base or radix with examples from the decimal, binary, octal, and hexadecimal number systems.
- Q5. Why is the binary number system used inside digital computers? Explain its advantages.
- Q6. Convert each of the following decimal numbers into binary, octal, and hexadecimal: (a) 56 (b) 128 (c) 247 (d) 1023 (e) 0.6875.
- Q7. Convert each of the following binary numbers into decimal, octal, and hexadecimal: (a) 1011011 (b) 11011011.101 (c) 10101010.11.
- Q8. Convert each of the following hexadecimal numbers into binary, octal, and decimal: (a) 7AF (b) 1B.C (c) FFFF.
- Q9. Perform the following binary additions: (a) $1011 + 1101$ (b) $11011 + 10110$ (c) $11111 + 1$.
- Q10. Perform the following binary subtractions: (a) $1101 - 1010$ (b) $11010 - 01101$ (c) $10000 - 00001$.
- Q11. Perform the following binary multiplications and divisions: (a) 1011×101 (b) $11011 \div 11$ (c) 1111×11 .
- Q12. Define 1's complement and 2's complement of a binary number. Find the 1's and 2's complements of 11001010 and 10001000.
- Q13. Subtract 01101_2 from 10110_2 using both 1's complement and 2's complement methods, and verify the answer.

- Q14. Define 9's complement and 10's complement of a decimal number. Find the 9's and 10's complements of 5832.
- Q15. Compare sign-magnitude, 1's complement, and 2's complement representations of signed binary numbers with examples.
- Q16. Define BCD code. Convert the decimal numbers 729 and 4096 into BCD form.
- Q17. What is Excess-3 code? Why is it called a self-complementing code? Convert the decimal number 596 into Excess-3 code.
- Q18. Define Gray code and its main applications. Convert the binary number 1011 into Gray code and verify the conversion.
- Q19. Differentiate between ASCII and Unicode. Why has Unicode replaced ASCII in most modern systems?
- Q20. What is a parity bit? Explain even and odd parity with examples and discuss its limitations.
- Q21. Briefly describe the Hamming code. How does it detect and correct errors in transmitted data?
- Q22. What is the difference between BCD and pure binary representation? Convert 853_{10} into both pure binary and BCD, and compare the results.
- Q23. Perform the following conversions: (a) 10110011_2 to Gray code (b) 1101_2 (Gray) back to binary (c) 10101_2 to Excess-3.